

Set it up on your laptop, and the auto-update plan

Part 1 gets the whole project running on your computer in about 20 minutes. Part 2 says honestly what auto-update already does today. Part 3 is the plan for making it configurable: what you and your customers can set, how it runs, what it costs, and what happens when something fails. Nothing in Part 3 is built yet; it waits for your answers in Part 4.

1. Set up on your laptop (exact steps)
2. Auto-update today: what already works
3. The plan: configurable auto-update
4. Decisions I need from you

1 · Set up on your laptop

Is everything on GitHub? Yes. All work is pushed to the branch `improve-editor-and-business` of `github.com/ahmmed-binas/brandme` (latest commit `a66af0e`). It is **not** merged into `main` yet, so you must clone that branch (step 2 does this).

STEP 1 · INSTALL THREE TOOLS (ONCE)

TOOL	WHERE	CHECK IT WORKS
Git	git-scm.com (Windows) · Mac: run <code>xcode-select --install</code>	<code>git --version</code>
Node.js 22 LTS	nodejs.org, the “LTS” download	<code>node -v</code> shows v22.x
Docker Desktop	docker.com/products/docker-desktop. Open it once and leave it running.	<code>docker --version</code>

On Windows, use **PowerShell** for every command below. On a Mac, use **Terminal**.

STEP 2 · DOWNLOAD THE PROJECT

```
git clone -b improve-editor-and-business https://github.com/ahmmed-binas/brandme.git
cd brandme
npm install
```

If the repository is private, Git asks you to sign in to GitHub; follow the browser prompt. `npm install` takes a few minutes.

STEP 3 · START THE DATABASE

One line (works in PowerShell and Terminal):

```
docker run -d --name formora-db -p 5432:5432 -e POSTGRES_PASSWORD=formora -e POSTGRES_DB=formora -v formora-db:/var/lib/postgresql/data postgres:16-alpine
```

If it says port 5432 is already in use, you already have PostgreSQL installed; change `-p 5432:5432` to `-p 5433:5432` and use port 5433 in step 4.

STEP 4 · SETTINGS FILE

Create a file called `.env.local` in the `brandme` folder with exactly this (use your own email):

```
DATABASE_URL=postgres://postgres:formora@localhost:5432/formora
APP_URL=http://localhost:3000
DEV_LOGIN=true
ADMIN_EMAILS=you@example.com
TEMPLATES_REQUIRE_APPROVAL=true
CRON_SECRET=local-cron-secret-change-me-123456
```

Then run `npx auth secret`. It adds a random `AUTH_SECRET` line to `.env.local` for you (if it only prints it, paste it in as `AUTH_SECRET=...`).

STEP 5 · START THE SITE

```
npm run dev
```

Open <http://localhost:3000>. The database tables are created automatically the first time.

STEP 6 · YOUR ACCOUNT AND A TEST RUN

1. Go to <http://localhost:3000/login>, scroll to **Local test account**, type the email from `ADMIN_EMAILS`, press **Sign in for testing**. No Google setup needed.
2. `/admin`: your tools and the go-live checklist (red items are expected on a laptop).
3. `/templates/review`: approve the templates you want customers to see.
4. `/templatechooser` → pick a template → **Use this template**. Click text in the preview to edit; the top bar shows *Saved to your account*. Reload: it comes back.
5. **Publish** → choose an address → visit `/p/your-address`.
6. Editor **Blog** tab → New post → Publish post → see `/p/your-address/blog`.
7. `/admin/journal` → write a Journal post → see `/blog`.
8. Sign out and sign in again with a different email to act as a normal customer.

OPTIONAL · TURN ON MORE FEATURES LOCALLY

FEATURE	ADD TO .ENV.LOCAL
AI assistant, CV import, research agent	<code>ANTHROPIC_API_KEY=...</code> (from platform.claude.com; costs a little per use)
Payments (test mode, no real money)	<code>STRIPE_SECRET_KEY=sk_test_...</code> and <code>STRIPE_WEBHOOK_SECRET</code> from <code>stripe listen --forward-to localhost:3000/api/webhooks/stripe</code>
Real emails	<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASSWORD</code> , <code>SMTP_FROM</code> (otherwise emails are printed in the terminal)
Run the hourly jobs once (auto-updates, reminders)	In a second terminal: <code>curl -X POST -H "Authorization: Bearer local-cron-secret-change-me-123456" http://localhost:3000/api/cron</code>

EVERYDAY COMMANDS

<code>npm run dev</code>	start the site (Ctrl+C to stop)
<code>docker start formora-db / docker stop formora-db</code>	start / stop the database (data is kept)
<code>git pull</code>	get my latest changes, then <code>npm install</code> again
<code>npm run build && npm start</code>	run the production version, exactly as on a server

If something goes wrong: “ECONNREFUSED 5432” means the database isn’t running (open Docker Desktop, run `docker start formora-db`). “Sign in for testing” missing means `DEV_LOGIN=true` or `APP_URL=http://localhost:3000` is missing; restart `npm run dev` after editing `.env.local`. A template shows 404: approve it at `/templates/review`.

2 · Auto-update today: what already works

Short answer: **partly built**. There is a working, scheduled auto-update, but its schedule and sources are fixed in code. Nobody can choose “check daily” or “watch my YouTube channel” yet.

WHAT	HOW OFTEN	PLANS	COST TO YOU
GitHub check: new public repositories become suggested projects	Weekly (fixed)	Trial, Pro, Premium	Free (no AI)
Research agent: Claude searches the web for the owner’s new role, talks, awards, press, launches, each with a source link	Every 90 days on Pro, every 30 days on Premium (fixed)	Pro, Premium	AI usage, paid from the plan’s monthly credits or the owner’s own API key; capped by <code>AI_DAILY_BUDGET_USD</code>
Weekly email digest: “You have 3 suggested updates”	Weekly	Anyone with new suggestions	Free
Review step: suggestions wait in the editor’s AI tab; nothing changes on the site until the owner clicks Add	–	All	–
On/off switch for automatic updates, plus “check now” buttons	–	All	–

What’s missing for your idea:

- Customers can’t choose **how often** or **which sources** are watched.
- You (admin) can’t change limits per plan without editing code (`lib/plans.ts`).
- Only two sources exist: GitHub and web research. Business owners also need their blog, YouTube, publications, reviews and so on.
- Everything needs manual approval; there’s no “apply safe updates for me”.
- No run history, so neither you nor the customer can see what was checked and why something failed.

3 · The plan: configurable auto-update

The idea in one sentence: **each customer chooses what to watch and how often (within limits their plan allows), you choose those limits per plan in the admin area, and an hourly job does the checks and turns findings into updates the customer approves, or applies safe ones automatically if they allow it.**



3.1 WHAT THE CUSTOMER SETS (EDITOR → AI TAB → “KEEP MY SITE UP TO DATE”)

SETTING	CHOICES
Sources to watch	A list of switches, each with a username or link (see 3.3)
How often	Daily · Weekly · Monthly · Off. Choices faster than their plan allows are shown locked with “Upgrade to check daily”
What happens to findings	Ask me first (default) · Add safe updates automatically : new blog post, new video, new GitHub project. Job changes and awards always ask, because getting those wrong is embarrassing
Email me	Each time · Weekly digest (default) · Never
History	“Last checked 2 hours ago · 3 new items” per source, and any error in plain words (“Your YouTube channel link isn’t public”)

3.2 WHAT YOU SET (NEW ADMIN PAGE /ADMIN/UPDATES)

SETTING PER PLAN	SUGGESTED DEFAULT: TRIAL / BASIC / PRO / PREMIUM
Fastest check allowed	Weekly / Monthly / Weekly / Daily
Sources allowed	Free sources / Free sources / Free sources + research / All
Research agent runs per month	0 / 0 / 1 / 4
Auto-apply allowed	No / No / Yes / Yes

Plus global switches: pause all auto-updates, pause research only (to stop AI spending instantly), and a daily AI budget. These are stored in the database, so changing them takes effect on the next run without a redeploy. The current values in `lib/plans.ts` become the defaults.

3.3 SOURCES

SOURCE	HOW	COST	STATUS
GitHub	Public API (exists today)	Free	Built
Their blog, Medium, Substack, any news feed	RSS/Atom feed: new posts become “Writing” highlights	Free	Phase 2
YouTube channel	The channel’s public RSS feed: new talks/videos	Free	Phase 2
Publications (researchers, doctors)	ORCID and Crossref public APIs by ORCID iD	Free	Phase 2
Their own Formora blog	New post → shown on the portfolio automatically (already happens)	Free	Built
Web research (new job, awards, press, talks)	Claude with web search (exists today), now on the customer’s chosen schedule within plan limits	AI credits	Built schedule: Phase 1
Behance / Dribbble	Behance has public RSS; Dribbble needs the owner to connect their account	Free	Phase 3
Google reviews (businesses)	Google Business Profile API; the owner must connect their Google business account; Google’s approval needed	Free	Phase 3
LinkedIn	LinkedIn gives no API for this. Instead: a monthly reminder email “Changed jobs? Upload your LinkedIn export and we’ll update your site”	Free	No API

3.4 HOW IT RUNS

- **New tables.** `update_sources` (one row per customer source: kind, username/link, on/off, how often, `next_run_at`, last result, error count); `update_runs` (history: when, what was found, errors, AI cost); `update_plan_limits` (your settings from 3.2). Existing GitHub usernames move into `update_sources` automatically.
- **Scheduler.** The existing hourly `/api/cron` picks up to N sources whose `next_run_at` has passed, using database row locks so two runs never check the same source. After each check it sets the next time from the chosen frequency, never faster than the plan allows. Free sources run in larger batches; research runs a few at a time.
- **Fetchers.** One small module per source kind with the same shape: given a source, return new items. Each item gets a fingerprint so the same post or project is never suggested twice (this already exists for GitHub).
- **Apply.** Approved or auto-applied updates go through the existing suggestion system, which edits the draft. The portfolio is then re-published automatically only if the owner turned on “publish automatic updates”; otherwise the editor shows “Unpublished changes”.
- **Undo.** Every auto-applied change is listed with an Undo button for 30 days.

3.5 WHEN THINGS FAIL (GRACEFUL DEGRADATION)

PROBLEM	WHAT HAPPENS
A source is down or a link is wrong	Retry later with growing gaps (1h, 6h, 1 day...). After 5 failures the source pauses and the owner gets one email explaining how to fix it. Other sources keep working.
Out of AI credits, or the daily AI budget is reached	Research waits for the next run; free sources continue. The owner sees “Research paused: no credits left” with a link to buy credits.
GitHub or another API rate-limits us	Stop that source kind for this run and continue next hour; batches are small to avoid this.
Plan downgraded or expired	Frequency drops to what the new plan allows; sources the plan doesn’t include pause (settings are kept, so upgrading turns them back on).
The cron job stops running	The /admin checklist shows “Scheduled jobs last ran 3 hours ago” in red.
A finding might be about someone else with the same name	Research findings marked low-confidence are never auto-applied and are labelled “Check this is you”.

3.6 COSTS

All non-AI sources (GitHub, RSS, YouTube, ORCID, Crossref) are free and light: a few hundred small requests an hour even with thousands of customers. The only real cost is the research agent, already metered and capped: each run is charged to the plan’s monthly credits (or the customer’s own API key), and `AI_DAILY_BUDGET_USD` is a hard ceiling for your key. Limiting research runs per plan (3.2) keeps cost predictable; I’ll show the measured average cost per run in /admin/updates so you can tune prices.

3.7 PHASES AND EFFORT

PHASE	WHAT YOU GET	EFFORT
1 • Configurable schedule	New tables; customer frequency and on/off per source; your /admin/updates plan limits; run history; failure handling; research and GitHub moved onto it	1–2 days
2 • More free sources	RSS/blog, Medium, Substack, YouTube, ORCID/Crossref; “add safe updates automatically”; Undo list; optional auto-publish	1–2 days
3 • Connected accounts	Dribbble, Google reviews (needs Google approval, can take weeks); LinkedIn export reminders	2–3 days + approval wait

Testing, as for everything so far: mock servers for every source, end-to-end tests for settings → scheduled run → suggestion → apply → undo, plus the failure cases in 3.5, run against a fresh database before anything is pushed.

4 • Decisions I need from you

1. **Plan limits:** are the defaults in 3.2 right? In particular, should Basic get any automatic checks?
2. **Auto-apply:** allowed at all, or always “ask me first”? (I recommend allowing it for blog posts, videos and GitHub projects only.)
3. **Auto-publish:** may automatic updates go live without the owner pressing Publish? (I recommend off by default, owner can turn on.)

4. **Sources:** are Phases 1 and 2 enough to start? Phase 3 needs Google's approval and is slower.

5. **Daily checks:** Premium only, or Pro too?

Next step: reply with your answers (or "use your recommendations") and I'll build Phase 1, test it, push it, and update this document.